

[Paper Review] You Only Look Once: Unified, Real-Time Object Detection

25기 분석

서영우

0. Abstract

- 기존의 객체 탐지 시스템이 분류기를 재사용했으나, YOLO의 경우 단일 신경망을 사용하여 전체 이미지에서 경계 상자와 클래스 확률을 직접 예측함으로써 배경에서 잘못된 탐지(FP)를 할 가능성이 낮아짐.

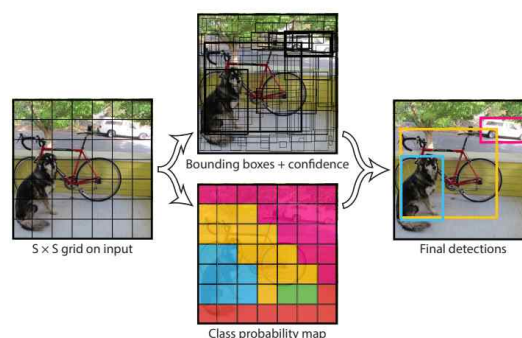
1. Introduction

- 기존 객체 탐지 시스템 : 분류기를 **재활용**하는 파이프라인을 사용하면서 최적화에 어려움을 겪음

*재활용 : Yolo 논문이 나오기 전, 이미지 내에서 물체가 있을 가능성이 있는 후보 영역(Box)을 수천 개 제안한 후, 분류기를 한번씩 저용해서 그 영역에 어떤 물체가 존재하는지 없는지 판단하는데, 이렇게 자신이 학습한 한 가지 역할만 유지한 채로 반복적으로 진행된다는 뜻

- Yolo : 객체 탐지 작업을 단일 회귀 문제로 재구성하고, 단일 Convolutional network가 여러 경계 상자와 class 확률을 동시에 예측하고, 전체 이미지를 보고 학습해서 최적화 진행.

2. Unified Detection



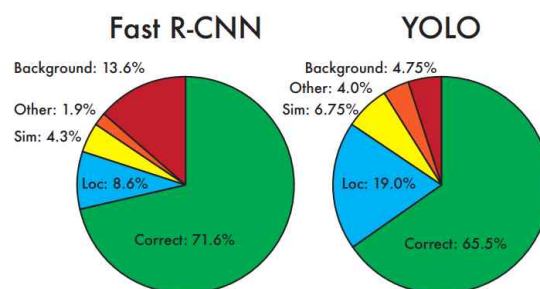
- 이전 객체 탐지 방식과 달리, 전체 이미지 특징을 사용해서 경계 상자를 예측한 후, 모든 클래스의 모든 경계 상자를 **동시에** 예측
- 입력 이미지를 $S \times S$ 로 나누고, 객체의 중심이 특정 그리드 셀에 속하면 객체를 탐지

- B개의 상자에 대한 신뢰도 점수, C개의 조건부 클래스 확률 $\Pr(Class_i|Object)$ 을 구했을 때, $S \times S \times (5B+C)$ 텐서로 인코딩
- GoogLeNet 모델을 기반으로 24 convolution layers와 2 fully connected layers로 구성
- Fast YOLO는 9개 convolution layer로 빠르게 작동하도록 설계

3. Training

- ImageNet 1000-class competition 데이터셋으로 사전학습한 후, PASCAL VOC 데이터셋으로 모델 훈련(탐지 목적)
- loss function으로 sum-squared error 사용
최적화 용이 but 정확도를 완전히 극대화할 수는 없다 **why?**
 - 1) 위치(localization) 오류와 분류(classification) 오류를 동등하게 취급하는데, 중요도가 같지 않을 수 있으므로
 - 2) 객체가 없는 그리드셀은 신뢰도 점수(confidence scores)를 0으로 만드는데, 이러면 기울기에 영향을 줘서 학습 불안정해짐
- > λ_{coord} , λ_{noobj} 두가지 매개 변수로 좌표 예측 손실 증가시키고, 객체가 없는 상자의 신뢰도 예측 손실 감소

4. Experiments & Results



- PASCAL VOC 2007 데이터셋 사용
- 속도랑 정확도 간의 trade off를 찾기 위해, YOLO(정확도 초점)랑 Fast YOLO(속도) + α 로 비교 (그래도 YOLO가 다른 모델 대비 빠르긴함)
- 위치 오류(localization errors)은 다른 모델에 비해 많았으나, (정확한 객체 위치 잡아내는 능력)
객체가 없는 배경을 객체로 잘못 탐지하는 배경 오류(background errors)는 3배 가까이 적음.
(객체가 없는 배경을 객체로 잘못 인식하는 가짜 탐지(FP)가 R-CNN보다 훨씬 적음)
- 모델이 훈련 과정에서 보지 못했던 새로운 종류의 데이터나 환경에서도 성능저하 다른 모델 대비 훨씬 적음 (**Generalizability** 우수)

5. Discussion/Conclusion

- 분류기 기반 접근 방식과 달리 탐지 성능에 직접적으로 관련된 손실 함수로 훈련
- 빠르고 견고한 객체 탐지가 필요한 애플리케이션에 적합
Ex) 자율주행 / 보안(CCTV) / 드론 등등

6. 기존 연구 대비 차별점

- Transformer 아키텍처 사용 (long range dependencies 해결)
- 단일 모델 구조 (입력 형태만 변환 -> 효율성/속도 향상)

[Paper Review] AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

25기 분석

서영우

0. Abstract

- Transformer 아키텍처를 CV 분야에 적용한 모델. 대규모 데이터로 사전 훈련을 하고 소규모 데이터로 Transfer learning을 진행했을 때, 상대적으로 우수한 결과를 얻음.
(우수하다.)

1. Introduction

- 자연어 처리 분야에서는 Self-Attention 기반 Transformer가 표준이 되었는데, CNN이 지배적인 CV분야에서 트랜스포머 모델 적용 시도
- 이미지를 patch로 분할하고, patch를 nlp의 token처럼 취급해서 input으로 제공
- 초기에는 이미지에 특화된 inductive bias가 없는 ViT가 성능이 안 좋았는데, 대규모 데이터셋으로 사전 학습하여 해결
 - > 개인적으로 어느 모델이나 대규모로 학습하면 좋은 결과를 얻는 것이 아닐까란 의문이
- 들긴 함.

2. Method

- 이미지를 2D patch sequence로 변환한 후 flattening하여 D차원의 잠재 벡터로 매핑 (Patch Embeddings)
- BERT의 class 토큰과 유사하게, 학습 가능한 classification token을 patch embedding sequence 앞에 추가해서 token 마지막 상태가 이미지 표현으로 사용되고, 위치 정보를 유지하기 위해 패치 임베딩에 위치 임베딩(position embeddings) 더해진다

ViT의 핵심 구조

1) 이미지 패치 시퀀스화

2D 이미지 데이터를 1D 시퀀스 데이터로 변환하는 과정.
이미지를 고정된 정사각형 패치들로 나누기
ex) 224X224, patch 크기 16X16 -> 196개의 patch 생성

2) Patch Embedding

각각의 패치는 픽셀들의 집합으로도 볼 수 있는데, 픽셀 값을 그대로 트랜스포머에 입력하는 것이 아니라, 각 패치를 NLP 기준으로 단어처럼 다루기 위해 **선형 변환**을 통해 D차원

의 벡터로 변환.

*선형 변환 : 벡터 공간을 다른 벡터 공간으로 보내는 함수로, 벡터의 덧셈과 스칼라 곱셈 연산을 보존하는 변환

3) Position Embedding

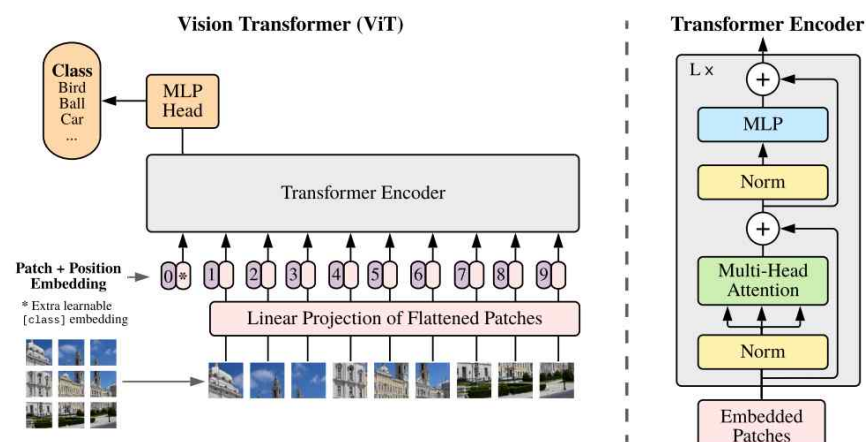
patch를 sequence로 나열하면 원래 공간 정보가 손실되는 걸 막기 위해 각 patch가 원래 이미지의 어느 위치에 있었는지에 대한 position embedding을 더함.

4) 분류 토큰 추가

BERT와 유사하게 모든 patch embedding sequence 맨 앞에 이미지 전체를 대표하는 역할을 하는 토큰을 추가함.

5) Transformer Encoder

‘분류토큰+패치 임베딩+위치 임베딩’의 시퀀스가 인코더의 input으로 대입



3. Experiments

- 표현 학습 능력(Representation learning capabilities)을 평가하기 위해 데이터셋의 크기를 기준으로 데이터셋을 사진 훈련하고, Adam 옵티마이저를 사용해서 훈련하고, finetuning은 SGD 사용 .

4. Discussion & Conclusion

- 초기 패치 추출 단계 외에는 inductive bias 도입 X
- 대신 이미지를 일련의 패치로 해석하여 표준 Transformer의 input으로 활용
- 기존 CV에 Self-attention을 적용하는 연구와 차별점
- CNN 의존성을 탈피하는 새로운 시도

5. 다뤄볼 수 있는 부분 & 의문점

본 페이퍼를 보면 대규모 데이터셋으로 사전 훈련할 경우 CNN보다 우수한 성능을 보여준다고 하는데, 모든 모델이 대규모 데이터셋을 선호하지 않나.. 적은 데이터에도 불구하고 좋은 성능을 보여주어야 맞는 것이 아닌가란 생각은 들지만.. 새로운 시도였다는 점에 의의를 둔다로 마무리 짓는 것이 맞다 판단.